

UNIWERSYTET RZESZOWSKI

WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH



Konrad Szyszlak

131524

Informatyka

Zintegrowany system smart parkingu z rozpoznawaniem tablic
rejestracyjnych

Praca Inżynierska

Praca wykonana pod kierunkiem

dr inż. Przemysław Pardel

Rzeszów 2026

*Serdecznie dziękuję dr inż. Przemysław Pardel za
opiekę merytoryczną nad pracą.
Dziękuję również wszystkim wspierającym osobom,
które poznałem w trakcie studiów*

WSTĘP	5
Wprowadzenie	5
Cel pracy	6
Problem badawczy i zakres pracy	6
Hipotezy badawcze	7
1. ANALIZA STANU TECHNIKI I ROZWIĄZAŃ RYNKOWYCH	7
1.1. Ewolucja systemów identyfikacji wizyjnej w infrastrukturze miejskiej	7
Metody detekcji i lokalizacji tablic rejestracyjnych	7
Algorytmy optycznego rozpoznawania znaków (OCR)	8
Wpływ warunków środowiskowych na akwizycję obrazu	8
1.2. Standardy komunikacyjne i scentralizowana architektura systemów	8
Architektura przetwarzania: Separacja warstwy sensora i logiki	8
Dobór protokołów sterowania: Analiza porównawcza REST vs MQTT	9
1.3. Szczegółowa analiza rozwiązań rynkowych	9
Komercyjne systemy klasy Enterprise	9
Rozwiązania typu Open Source i niskobudżetowe platformy wbudowane	10
1.4. Identyfikacja luki badawczej i uzasadnienie projektu	11
2. TEORETYCZNE PODSTAWY ZASTOSOWANYCH TECHNOLOGII	11
2.1. Architektury konwolucyjnych sieci neuronowych w procesach detekcji (YOLO)	12
2.2. Sieci rekurencyjne i LPRNet w predykcji ciągów alfanumerycznych	13
2.3. Protokół MQTT i mechanizmy QoS w aplikacjach IoT	13
2.4. Algorytmy adaptacyjnej poprawy jakości obrazu (CLAHE)	14
3. PROJEKT I IMPLEMENTACJA SYSTEMU	15
3.1. Założenia projektowe i diagram architektury sprzętowo-programowej	16
3.2. Konfiguracja środowiska serwerowego na platformie Debian Server	17
3.3. Opracowanie schematu i wdrożenie relacyjnej bazy danych PostgreSQL	17
3.4. Akwizycja strumienia wideo i implementacja preprocessingu obrazu w środowisku Python	18
3.5. Integracja modeli głębokiego uczenia dla detekcji i odczytu tablic	19
3.6. Budowa warstwy sprzętowej IoT i programowanie mikrokontrolera ESP32	19
3.7. Konfiguracja brokera MQTT i implementacja komunikacji asynchronicznej	19
3.8. Implementacja logiki biznesowej i webowego interfejsu administratora	19
4. WYNIKI BADAŃ I TESTY WYDAJNOŚCIOWE	24
4.1. Definicja scenariuszy oraz środowiska testowego	24
4.2. Skuteczność detekcji i odczytu tablic w warunkach pełnego oświetlenia	24
4.3. Analiza wpływu algorytmu CLAHE na poprawność odczytu w warunkach nocnych	24
4.4. Pomiary opóźnień systemu – latencja komunikacji	24
4.5. Stabilność działania w architekturze scentralizowanej	24

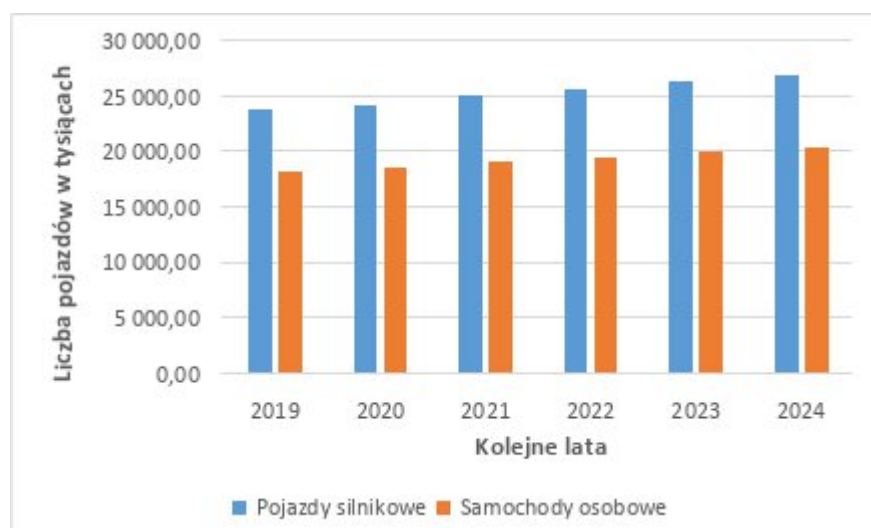
ZAKOŃCZENIE I WNIOSKI KOŃCOWE	25
BIBLIOGRAFIA	26
WYKAZ FOTOGRAFII	28
WYKAZ RYSUNKÓW	29
WYKAZ TABEL	30
WYKAZ WYKRESÓW	31
STRESZCZENIE	32

WSTĘP

Wprowadzenie

Współczesna infrastruktura miejska generuje coraz większą liczbę danych dotyczącą ruchu pojazdów[1], według Raportu PZPM 2025/2026 liczba pojazdów silnikowych w Polsce wzrosła z 23,8 mln w 2019 r. do 27,0 mln w 2024 r.[2], dane te wymagają automatyzacji w celu zachowania płynności i bezpieczeństwa. Tradycyjne systemy kontroli dostępu, oparte na fizycznych biletach, kartach zbliżeniowych bądź też pilotach radiowych, stają się niewydolne w obliczu tego ruchu. Generują one korki przy punktach wjazdowych i wyjazdowych[1] jak i również wysokie koszty utrzymania terminali oraz dystrybucji nośników uprawnień. Rozwój Internetu Rzeczy (IoT), miniaturyzacja elektroniki oraz spadek kosztów układów obliczeniowych umożliwiły tworzenie nowoczesnych, zautomatyzowanych systemów sterowania ruchem. Integracja kamer IP z algorytmami widzenia komputerowego oraz rozproszonymi urządzeniami wykonawczymi pozwala na centralizację danych i eliminację silosów informacyjnych, co znacząco zwiększa efektywność zarządzania infrastrukturą parkingową. W tym kontekście rośnie zapotrzebowanie na inteligentne systemy[1], które potrafią autonomicznie identyfikować pojazdy, weryfikować ich uprawnienia oraz sterować elementami fizycznymi w czasie rzeczywistym, bez udziału człowieka.

Niniejsza praca przedstawia projekt i implementację prototypu zintegrowanego systemu Smart Parking, wykorzystującego techniki głębokiego uczenia do detekcji i rozpoznawania tablic rejestracyjnych oraz architekturę IoT do sterowania urządzeniami wykonawczymi. Celem pracy jest wykazanie, że przy użyciu ogólnodostępnych, niskokosztowych komponentów możliwe jest stworzenie rozwiązania o funkcjonalności porównywalnej z komercyjnymi systemami klasy Enterprise.



Rys. 1. Park pojazdów silnikowych w Polsce w latach 2019–2024 (opracowanie własne na podstawie: PZPM, Raport Branży Motoryzacyjnej 2025/2026)[2].

Cel pracy

Głównym celem pracy jest zaprojektowanie i budowa funkcjonalnego prototypu inteligentnego systemu zarządzania parkingiem z wykorzystaniem niskokosztowych komponentów rynkowych (COTS), który w sposób autonomiczny realizuje pełny cykl obsługi pojazdu, począwszy od detekcji wjazdu po weryfikację uprawnień i sterowanie elementem wykonawczym (w przypadku prototypu serwomechanizmem). System ma za zadanie automatyzację procesu kontroli dostępu w oparciu o wizyjną identyfikację tablic rejestracyjnych (ALPR). Projekt ma dowieść, że możliwe jest stworzenie wysokiej klasy systemu analitycznego bez konieczności inwestowania w drogie, zamknięte rozwiązania klasy Enterprise.

Problem badawczy i zakres pracy

Głównym problemem technicznym rozwiązywanym w ramach pracy jest zapewnienie wysokiej skuteczności rozpoznawania znaków w niekontrolowanych i zmiennych warunkach oświetleniowych (niedoświetlenie nocne, odbłaski od reflektorów) przy użyciu standardowych, budżetowych kamer IP obsługujących protokół RTSP. Kolejnym wyzwaniem jest zaprojektowanie architektury scentralizowanej, która pozwoli na odseparowanie warstwy sensora od logiki obliczeniowej, zapewniając niską latencję i stabilność komunikacji w modelu sterowanym zdarzeniami.

Zakres pracy obejmuje:

- Implementację potoku wizyjnego (Vision Engine) wykorzystującego sieci neuronowe YOLO do detekcji oraz LPRNet do rozpoznawania sekwencji znaków.
- Opracowanie algorytmów preprocessingu obrazu z wykorzystaniem techniki CLAHE w celu eliminacji szumów oświetleniowych.
- Budowę warstwy sterowania IoT opartej na mikrokontrolerze ESP32 i protokole MQTT do zarządzania modelem szlabanu.
- Zaprojektowanie i wdrożenie relacyjnej bazy danych PostgreSQL do zarządzania rejestrem uprawnionych pojazdów i ewidencji zdarzeń.
- Stworzenie webowego panelu administratora do monitorowania stanu parkingu i zarządzania bazą użytkowników.
- Zaprojektowanie i budowę sprzętowego symulatora terminala płatniczego (klasy Nayax[21]), działającego w oparciu o mikrokontroler ESP32 oraz czytnik NFC.

Z zakresu pracy wyłączono realizację rzeczywistych transakcji finansowych. System symuluje proces płatności[21] poprzez odczyt i pomniejszanie wirtualnego salda

zapisanego fizycznie w pamięci karty zbliżeniowej. Po udanej operacji symulator komunikuje się z systemem głównym poprzez wysłanie odpowiedniej sekwencji impulsów elektrycznych, dokładnie odwzorowując zachowanie prawdziwego urządzenia rynkowego. Zapewnia to pełną kompatybilność sprzętową (ang. *plug-and-play*) – zastąpienie symulatora komercyjnym terminalem będzie możliwe wyłącznie poprzez przełączenie przewodów sygnałowych, bez ingerencji w kod źródłowy systemu.

Hipotezy badawcze

W procesie projektowym postawiono następujące hipotezy:

1. Zastosowanie adaptacyjnego wyrównywania histogramu (CLAHE) pozwala na skuteczne rozpoznawanie tablic rejestracyjnych przez sieć LPRNet nawet w trudnych warunkach nocnych przy użyciu tanich kamer IP.
2. Scentralizowana architektura serwerowa umożliwia jednoczesną obsługę wielu punktów wjazdowych bez zauważalnego wzrostu opóźnień (latencji) w sterowaniu infrastrukturą fizyczną.
3. Wykorzystanie protokołu MQTT z mechanizmami QoS oraz Last Will and Testament (LWT) gwarantuje wyższą niezawodność i stabilność sterowania szlabanem niż tradycyjne odpytywanie w architekturze REST API.

1. ANALIZA STANU TECHNIKI I ROZWIĄZAŃ RYNKOWYCH

Tradycyjny, sztywny podział na parkingi zamknięte i ogólnodostępne stopniowo zanika na rzecz inteligentnych stref zarządzania ruchem[1]. Niniejszy rozdział stanowi szczegółowy przegląd ewolucji systemów wizyjnych, analizę standardów komunikacyjnych oraz zestawienie dostępnych na rynku rozwiązań parkingowych. Na podstawie tej analizy zidentyfikowano lukę technologiczną, która stała się bezpośrednim uzasadnieniem dla realizacji niniejszego projektu.

1.1. Ewolucja systemów identyfikacji wizyjnej w infrastrukturze miejskiej

Metody detekcji i lokalizacji tablic rejestracyjnych

Początki systemów automatycznego rozpoznawania tablic rejestracyjnych (ALPR) opierały się na klasycznych metodach przetwarzania obrazu i tak zwanej ręcznej ekstrakcji cech. Wykorzystywano algorytmy oparte na kaskadach Haara[3] lub histogramach zorientowanych gradientów (HOG)[5]. Metody te, mimo niskiego

zapotrzebowania na moc obliczeniową, cechowały się niską skutecznością w niekontrolowanych warunkach środowiskowych. Były one skrajnie wrażliwe na zmiany oświetlenia, cienie, zabrudzenia tablic oraz perspektywę, pod jaką pojazd podjeżdżał do punktu kontrolnego.

Przełom technologiczny nastąpił wraz z upowszechnieniem głębokich sieci neuronowych (CNN). Pierwsze generacje detektorów obiektów, takie jak Faster R-CNN, należały do grupy modeli dwustopniowych. Oferowały one wysoką precyzję, jednak proces generowania propozycji regionów wprowadzał zbyt duże opóźnienia, aby możliwe było ich zastosowanie na standardowych procesorach w czasie rzeczywistym. Odpowiedzią na te ograniczenia stały się detektory jednostopniowe, takie jak wykorzystywana w niniejszym projekcie architektura YOLO. Traktuje ona proces detekcji jako pojedynczy problem regresji, co pozwala na drastyczne zwiększenie szybkości działania przy zachowaniu wysokiej odporności na zmienne warunki pogodowe i oświetleniowe.

Algorytmy optycznego rozpoznawania znaków (OCR)

Równolegle ewoluowały techniki odczytu znaków (OCR). Klasyczne podejścia, wymagające wstępnej segmentacji (dzielenia tablicy na pojedyncze litery), okazywały się zawodne w przypadku obecności hologramów czy śrub montażowych. Współczesne standardy, reprezentowane przez sieci typu LPRNet, analizują obraz tablicy jako nierozzerwalną całość, co pozwala na poprawną identyfikację numeru nawet przy częściowym przysłonięciu lub zniekształceniu znaków.

Wpływ warunków środowiskowych na akwizycję obrazu

Skuteczność algorytmów ALPR jest ściśle skorelowana z jakością danych wejściowych. W warunkach nocnych automatyka kamer IP często wydłuża czas naświetlania, co prowadzi do rozmycia obrazu. Równie istotne jest oślepienie sensora przez reflektory pojazdu. W celu redukcji tych zjawisk zastosowano techniki preprocessingu, w szczególności adaptacyjne wyrównywanie histogramu (CLAHE), co pozwala na odzyskanie czytelności znaków z obszarów niedoświetlonych.

1.2. Standardy komunikacyjne i scentralizowana architektura systemów

Architektura przetwarzania: Separacja warstwy sensora i logiki

Współczesne systemy wizyjne odchodzą od modelu zintegrowanego (gdzie kamera sama analizuje obraz) na rzecz architektury scentralizowanej. W tym modelu kamera IP pełni rolę pasywnego sensora dostarczającego strumień wideo za pomocą

protokołu RTSP (ang. Real Time Streaming Protocol). Cała logika obliczeniowa oraz wnioskowanie sieci neuronowych odbywa się na dedykowanym serwerze lokalnym. Taka separacja warstwy sprzętowej od programowej pozwala na łatwą modernizację algorytmów bez konieczności kosztownej wymiany kamer.

Dobór protokołów sterowania: Analiza porównawcza REST vs MQTT

Kluczowym elementem systemów Smart Parking jest również warstwa sterowania urządzeniami wykonawczymi (szlabanami). Analiza standardów komunikacyjnych wykazuje przewagę protokołów zorientowanych na zdarzenia, takich jak MQTT, nad klasyczną architekturą REST API. Protokół MQTT, pracujący w modelu publikacja-subskrypcja, zapewnia natychmiastową reakcję sterownika na komendę z serwera, minimalizując latencję systemu. Jest to rozwiązanie optymalne dla urządzeń o ograniczonej mocy obliczeniowej, zapewniające stabilność połączenia nawet w środowiskach o dużej gęstości sygnałów bezprzewodowych.

1.3. Szczegółowa analiza rozwiązań rynkowych

Rynek systemów identyfikacji pojazdów charakteryzuje się silną polaryzacją[1]. Z jednej strony dominują kosztowne, zamknięte ekosystemy producentów sprzętu wizyjnego, z drugiej – fragmentaryczne rozwiązania programowe o otwartym kodzie źródłowym. Poniższa analiza wskazuje na brak elastycznych rozwiązań pośrednich, które łączyłyby niski koszt wdrożenia z możliwością pełnej kustomizacji logiki biznesowej.

Komercyjne systemy klasy Enterprise

Analiza czołowych rozwiązań dostępnych na rynku wykazuje silną tendencję do oferowania zamkniętych ekosystemów, w których warstwa programowa jest ściśle powiązana z dedykowanym sprzętem. Przegląd wybranych ofert producentów potwierdza, że dominującym modelem biznesowym jest wycena indywidualna, co uniemożliwia proste szacowanie kosztów wdrożenia dla mniejszych inwestorów.

Reprezentantem podejścia "End-to-End" jest firma Green Center[9]. Oferuje ona kompletny system parkingowy (szlabany, terminale wjazdowe, kasy automatyczne) oparty na autorskim oprogramowaniu. Jest to typowy przykład architektury zamkniętej, gdzie klient zobligowany jest do zakupu całej infrastruktury od jednego dostawcy, bez możliwości integracji kamer czy szlabanów firm trzecich. Producent nie udostępnia publicznie cennika, opierając się wyłącznie na modelu projektowym.

Podobną filozofię prezentuje firma Smart2Park[11] z systemem SWMP. Rozwiązanie to jest wysoce zaawansowane – oferuje nie tylko kontrolę dostępu, ale również system nawigacji do wolnego miejsca oraz wyświetlacze strefowe. Choć producent podaje ceny elementów wykonawczych (szlabany), koszt samego systemu zarządzającego i sensorów pozostaje ukryty i podlega indywidualnej wycenie. System ten, ze względu na stopień skomplikowania i konieczność instalacji dedykowanych czujników zajętości, dedykowany jest głównie dla dużych galerii handlowych, a nie elastycznych stref parkingowych.

Z kolei firma CarFlow[12] System oferuje szeroki wachlarz produktów do automatyzacji, jednak podobnie jak konkurencja, nie udostępnia transparentnego cennika wdrożeniowego, co sugeruje model biznesowy nastawiony na dużych klientów korporacyjnych i kontrakty B2B.

Inne podejście prezentuje City Parking Group z produktem Parkomatic[10]. System ten eliminuje tradycyjne bilety na rzecz identyfikacji tablic (ANPR), a płatności realizowane są przez dedykowaną aplikację mobilną. Mimo nowoczesnego podejścia (brak szlabanów w niektórych konfiguracjach), jest to system zamknięty, uzależniający zarządcę terenu od operatora płatności i jego aplikacji, bez możliwości łatwej zmiany dostawcy usług finansowych.

W sektorze zarządzania biurowego (Smart Office) funkcjonują rozwiązania takie jak SmartOffice.expert[13] (moduł Smart Park). Skupiają się one na warstwie rezerwacji miejsc i obsłudze gości z poziomu aplikacji, jednak nie posiadają własnego, zintegrowanego modułu ANPR do fizycznego sterowania szlabanem, zamiast tego korzysta z kodów QR

Uzupełnieniem rynku są rozwiązania koncernów takich jak Asseco Data Systems[14] czy SPIE[15]. Systemy te (np. Asseco Strefy Parkowania) koncentrują się na detekcji zajętości miejsc w skali makro (Smart City) za pomocą kamer monitoringu. Oferta firmy SPIE, choć koncepcyjnie zbliżona do założeń niniejszej pracy, jest prezentowana w sposób ogólnikowy, bez udostępnienia specyfikacji technicznej czy widełek cenowych, co jest charakterystyczne dla rozwiązań wdrażanych w modelu przetargowym.

Rozwiązania typu Open Source i niskobudżetowe platformy wbudowane

Alternatywą dla systemów komercyjnych są projekty o otwartym kodzie źródłowym oraz usługi chmurowe, jednak one również posiadają istotne ograniczenia techniczne. Historycznym standardem w środowisku Open Source była biblioteka OpenALPR[16]. Analiza publicznego repozytorium wskazuje jednak, że wersja otwarta

nie jest już aktywnie rozwijana i bazuje na starszych generacjach algorytmów, co skutkuje obniżoną skutecznością detekcji w trudnych warunkach oświetleniowych.

Współczesną alternatywą są frameworki takie jak Nomeroff Net[17], wykorzystujące nowoczesną architekturę YOLO. Należy jednak zaznaczyć, że jest to wyłącznie biblioteka programistyczna (SDK), służąca do ekstrakcji danych z obrazu. Nie stanowi ona kompletnego produktu – brakuje w niej warstwy logiki biznesowej, takiej jak baza danych użytkowników, moduł płatności czy obsługa protokołów sterowania. Wykorzystanie tego narzędzia wymagałoby więc zaprojektowania całego ekosystemu zarządczego, co potwierdza celowość realizacji niniejszego projektu.

W segmencie niskobudżetowym popularność zyskują usługi SaaS[18] (np. Plate Recognizer), oferujące rozpoznawanie tablic przez API. Choć odciąża to lokalny procesor, w systemach sterowania ruchem wprowadza krytyczne opóźnienia sieciowe (latencję) oraz ryzyko paraliżu wjazdu w przypadku awarii łącza internetowego. Model ten generuje również stałe koszty operacyjne, co w długim okresie podważa sens ekonomiczny rozwiązania budżetowego.

1.4. Identyfikacja luki badawczej i uzasadnienie projektu

Przeprowadzona analiza wykazuje istnienie luki technologicznej w segmencie rozwiązań typu „Middle-ware”. Brakuje oprogramowania pośredniczącego[1], które byłoby agnostyczne sprzętowo (obsługiwałoby dowolne kamery IP i tanie mikrokontrolery) i jednocześnie dostarcza kompletną logikę zarządczą znaną z drogich systemów Enterprise.

Uzasadnieniem dla realizacji niniejszego projektu jest potrzeba stworzenia systemu, który połączy ekonomiczną efektywność ogólnodostępnych komponentów (COTS) z zaawansowaną analityką obrazu opartą na głębokim uczeniu. Projektowany system wypełnia tę lukę, oferując lokalne przetwarzanie danych (gwarantujące prywatność i brak opóźnień) oraz elastyczny silnik reguł biznesowych, umożliwiając wirtualizację procesów płatności i bezobsługowe zarządzanie dostępem niezależnie od skali obiektu.

2. TEORETYCZNE PODSTAWY ZASTOSOWANYCH TECHNOLOGII

Zaprojektowanie wydajnego systemu kontroli dostępu, opartego na analizie obrazu w czasie rzeczywistym i scentralizowanej architekturze, wymaga integracji nowoczesnych algorytmów z niezawodnymi protokołami komunikacyjnymi. W niniejszym rozdziale przedstawiono teoretyczne inżynierskie podstawy kluczowych technologii wybranych do realizacji projektu. Omówiono architekturę głębokich sieci

neuronowych odpowiedzialnych za detekcję (YOLO) i optyczne rozpoznawanie znaków (LPRNet), mechanizmy protokołu MQTT zapewniające sprzętową niezawodność transmisji, a także algorytm adaptacyjny CLAHE, wykorzystany w procesie sprzętowo-programowej poprawy jakości akwizycji obrazu.

2.1. Architektury konwolucyjnych sieci neuronowych w procesach detekcji (YOLO)

Podstawą współczesnych systemów widzenia komputerowego są konwolucyjne sieci neuronowe (ang. Convolutional Neural Networks, CNN). Ich architektura pozwala na automatyczną ekstrakcję cech z obrazu wejściowego poprzez operację splotu (konwolucji) z wykorzystaniem przesuwających się filtrów. Dzięki warstwom łączącym (ang. pooling) sieci te redukują wymiarowość danych, zachowując jednocześnie odporność na przesunięcia przestrzenne obiektów w kadrze.

Historycznie, proces detekcji obiektów w głębokim uczeniu opierał się na architekturach dwustopniowych (takich jak R-CNN). W pierwszym kroku algorytm generował zbiór potencjalnych regionów zawierających obiekt (ang. Region Proposal), a w drugim dokonywał ich klasyfikacji. Podejście to, choć cechujące się wysoką precyzją, wprowadzało znaczny narzut obliczeniowy. Dyskwalifikowało to takie modele w aplikacjach wymagających niskiej latencji, takich jak systemy parkingowe analizujące strumień wideo na żywo.

Przełomem w optymalizacji czasu przetwarzania okazała się architektura YOLO[4][20] (ang. You Only Look Once), która zdefiniowała problem detekcji jako pojedynczy problem regresji. Algorytm dzieli obraz wejściowy na regularną siatkę komórek (ang. grid). Jeśli środek geometryczny poszukiwanego obiektu (w tym przypadku tablicy rejestracyjnej) znajduje się wewnątrz danej komórki, staje się ona odpowiedzialna za jego detekcję. Każda komórka predykuje współrzędne ramek otaczających (ang. bounding boxes) oraz poziom pewności (ang. confidence score), określający prawdopodobieństwo wystąpienia obiektu oraz precyzję dopasowania samej ramki.

W projektowanym systemie wykorzystano współczesną implementację tej architektury, zoptymalizowaną pod kątem wydajności. Jednostopniowy charakter sieci YOLO pozwala na wnioskowanie (ang. inference) w czasie rzeczywistym. Szybkość ta jest krytyczna w przyjętej scentralizowanej architekturze serwerowej, gdzie jedna jednostka centralna musi asynchronicznie dekodować i przetwarzać strumień wideo (RTSP) z kilku kamer wjazdowych jednocześnie, nie powodując zatorów w kolejce sprzętowej.

2.2. Sieci rekurencyjne i LPRNet w predykcji ciągów alfanumerycznych

Po pomyślnym wyodrębnieniu i wycięciu obszaru tablicy rejestracyjnej przez detektor YOLO, proces przechodzi do fazy optycznego rozpoznawania znaków (OCR). Tradycyjne metody klasyfikacji tekstu z obrazu wymagają uprzedniej, rygorystycznej segmentacji. Oznacza to konieczność odizolowania każdego znaku przed jego identyfikacją. W przypadku polskich tablic rejestracyjnych techniki te są wysoce podatne na błędy wywoływane przez fizyczne artefakty takie jak: naklejki legalizacyjne, śruby montażowe, zabrudzenia czy cienie rzucane przez ramkę, które klasyczne algorytmy segmentacyjne błędnie interpretują jako fragmenty liter.

Rozwiązaniem tego problemu technicznego jest zastosowanie architektury LPRNet[6][20] (ang. License Plate Recognition Network), zaprojektowanej specjalnie do odczytu tablic w trybie bezsegmentacyjnym (ang. segmentation-free). Model ten całkowicie eliminuje etap dzielenia obrazu na pojedyncze znaki, traktując wycięty tensor jako integralną sekwencję. LPRNet charakteryzuje się relatywnie niewielką wagą (ang. lightweight architecture), co pozwala na jej implementację bez konieczności stosowania kosztownych akceleratorów GPU klasy serwerowej.

Kluczowym inżynierskim mechanizmem w tej architekturze jest zastosowanie funkcji straty CTC (ang. Connectionist Temporal Classification). Mechanizm ten pozwala na matematyczne dopasowanie sekwencji przewidywanych znaków do sekwencji wzorcowej, nawet jeśli szerokość fizyczna (reprezentowana przez liczbę pikseli) poszczególnych liter i cyfr na obrazie jest nieregularna. Sieć najpierw przetwarza obraz przez warstwy konwolucyjne, ekstrahując mapę cech przestrzennych, a następnie za pomocą mechanizmów sekwencyjnych generuje najbardziej prawdopodobny ciąg alfanumeryczny. To podejście sprawia, że system jest sprzętowo i programowo uodporniony na odczyt zniekształceń tła i hologramów.

2.3. Protokół MQTT i mechanizmy QoS w aplikacjach IoT

Fizyczna realizacja kontroli dostępu w systemach parkingowych wymaga niezawodnej komunikacji między centralną jednostką analityczną a urządzeniami brzegowymi (elementami wykonawczymi), takimi jak sterowniki szlabanów czy tablice informacyjne LED. W tradycyjnych systemach webowych dominującym standardem wymiany danych jest architektura REST oparta na protokole HTTP[19]. Komunikacja ta ma jednak charakter bezstanowy i synchroniczny (klient-serwer), co w przypadku urządzeń wbudowanych o ograniczonych zasobach obliczeniowych – takich jak mikrokontrolery z rodziny ESP32 – rodzi istotne problemy wydajnościowe. Wymusza ona stosowanie mechanizmu aktywnego odpytywania (ang. polling), w którym

sterownik musi cyklicznie inicjować połączenie z serwerem, aby sprawdzić, czy pojawiła się nowa komenda otwarcia bariery. Generuje to zbędny narzut sieciowy i obciąża procesor urządzenia brzegowego.

Z tego względu w projektowanym systemie jako wiodący standard transmisyjny wybrano protokół MQTT[19] (ang. Message Queuing Telemetry Transport). Jest to asynchroniczny protokół warstwy aplikacji, zaprojektowany natywnie dla środowisk Internetu Rzeczy (IoT), działający w modelu publikacja-subskrypcja (ang. publish-subscribe) z wykorzystaniem stałego połączenia TCP/IP. Środowiskiem pośredniczącym w komunikacji jest tak zwany broker wiadomości.

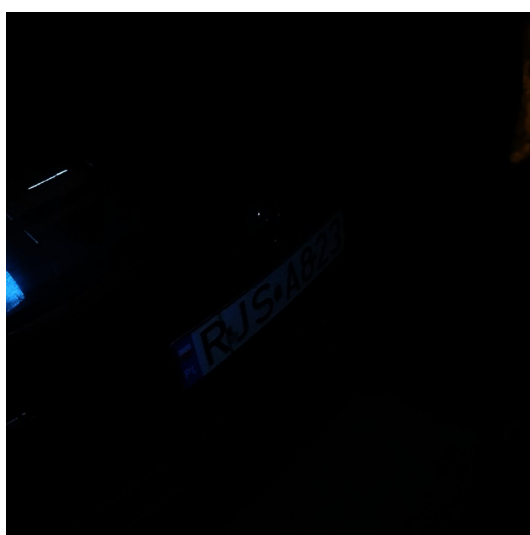
Z punktu widzenia inżynierii systemów niezawodnych, kluczową zaletą MQTT są wbudowane na poziomie protokołu mechanizmy gwarancji dostarczenia pakietów, określane jako QoS (ang. Quality of Service)[8]. Zastosowanie odpowiedniego poziomu QoS gwarantuje sprzętowo, że krytyczny sygnał sterujący (np. impuls otwierający szlaban) dotrze do mikrokontrolera nawet w przypadku chwilowej degradacji łącza bezprzewodowego, bez konieczności implementacji skomplikowanej logiki ponawiania żądań w kodzie aplikacji. Dodatkowo, protokół oferuje mechanizm LWT (ang. Last Will and Testament), który pozwala centralnemu serwerowi na natychmiastowe wykrycie awarii zasilania lub utraty łączności ze sterownikiem szlabanu, co jest kluczowe dla monitorowania stanu infrastruktury parkingu w czasie rzeczywistym.

2.4. Algorytmy adaptacyjnej poprawy jakości obrazu (CLAHE)

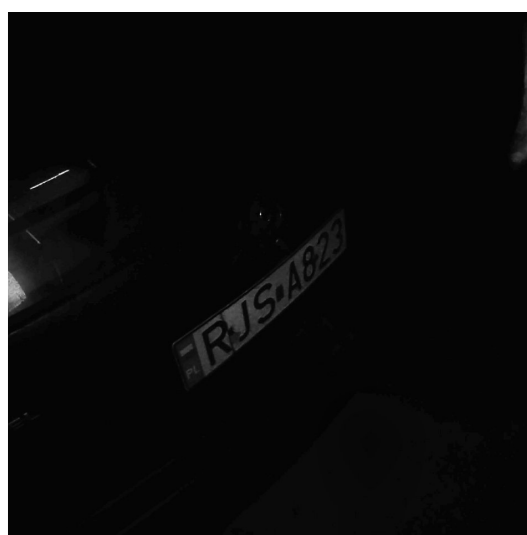
Skuteczność algorytmów rozpoznawania tablic rejestracyjnych (ALPR) jest bezpośrednio skorelowana z jakością i czytelnością danych wejściowych. Wykorzystanie ekonomicznych, uniwersalnych kamer IP (klasy COTS) wiąże się z kompromisami sprzętowymi, takimi jak niższa dynamika tonalna matrycy (HDR) czy ograniczona czułość sensora w warunkach nocnych. Zjawiska fizyczne, takie jak punktowe oślepianie obiektywu przez światła mijania pojazdu oraz głębokie cienie w obszarze zderzaka, prowadzą do utraty kluczowych detali, uniemożliwiając poprawną predykcję znaków przez sieć LPRNet.

W celu skompensowania tych ograniczeń sprzętowych na drodze programowej, potok przetwarzania obrazu (ang. Vision Pipeline) został rozbudowany o moduł preprocessingu wykorzystujący algorytm CLAHE[7] (ang. Contrast Limited Adaptive Histogram Equalization). W odróżnieniu od globalnego wyrównywania histogramu, które analizuje i modyfikuje piksele na podstawie całego kadru (co często prowadzi do przepaleń obrazu i wzmocnienia szumu w tle), metoda CLAHE działa lokalnie.

Algorytm dzieli klatkę wideo na mniejsze, regularne regiony (ang. tiles), a następnie optymalizuje rozkład jasności niezależnie dla każdego z nich. Kluczowym inżynierskim parametrem tego rozwiązania jest mechanizm limitowania kontrastu (ang. clip limit). Zapobiega on nadmiernemu wzmacnianiu szumu cyfrowego w jednorodnych, ciemnych obszarach obrazu. Zastosowanie adaptacyjnej korekcji pozwala na „wyciągnięcie” struktury znaków alfanumerycznych z niedoświetlonych fragmentów tablicy oraz zredukowanie efektu odbłasków od powłok odbłaskowych, znacząco podnosząc współczynnik poprawnych odczytów (ang. accuracy) w trudnych warunkach środowiskowych bez konieczności kosztownej modernizacji infrastruktury optycznej.



(a) Obraz oryginalny (Low-Light)



(b) Obraz po przetworzeniu

CLAHE

Rys. 2. Wpływ preprocessingu na czytelność tablicy rejestracyjnej w trudnych warunkach oświetleniowych. Źródło: Opracowanie własne.

3. PROJEKT I IMPLEMENTACJA SYSTEMU

W rozdziale tym zawarto szczegółową dokumentację techniczną zaprojektowanego prototypu systemu. Przedstawiono w nim architekturę sprzętowo-programową, proces konfiguracji środowiska serwerowego oraz układów wbudowanych, a także projekt i implementację logiki biznesowej sterującej procesem identyfikacji wizyjnej.

3.1. Założenia projektowe i diagram architektury sprzętowo-programowej

System został zaprojektowany w oparciu o scentralizowaną architekturę warstwową (ang. Centralized Layered Architecture) z elementami architektury sterowanej zdarzeniami (ang. Event-Driven Architecture). Takie podejście pozwala na odseparowanie warstwy akwizycji danych od logiki decyzyjnej, co zapewnia wysoką elastyczność i skalowalność prototypu.

Struktura warstwowa Struktura systemu dzieli się na pięć kluczowych poziomów funkcjonalnych:

- **Warstwa Akwizycji (Perception Layer):** Składa się z rozproszonych kamer IP obsługujących protokół RTSP. Kamery pełnią rolę pasywnych sensorów, dostarczając surowy strumień wideo do jednostki centralnej bez obciążania ich lokalną analityką.
- **Warstwa Przetwarzania Wizyjnego (Vision Processing Layer):** Stanowi „serce” systemu, zaimplementowane w środowisku Python na serwerze z systemem Debian. Wykorzystuje ona potok przetwarzania (Vision Pipeline) obejmujący moduły:
 - Preprocessing: Adaptacyjne wyrównywanie histogramu (CLAHE).
 - Inference: Detekcja obiektów (YOLO) oraz rozpoznawanie sekwencji znaków (LPRNet).
- **Warstwa Logiki Biznesowej i Zarządzania Dane (Core Logic & Data Layer):** Odpowiada za procesy autoryzacji i zarządzanie stanem systemu. Opiera się na relacyjnej bazie danych PostgreSQL, która przechowuje białą listę (Whitelist), taryfy (Tariffs) oraz ewidencję zdarzeń (ParkingEvents).
- **Warstwa Komunikacyjna (Communication Layer):** Wykorzystuje broker wiadomości MQTT jako szynę danych. Zastosowanie modelu publikacja-subskrypcja (Pub/Sub) pozwala na asynchroniczne przesyłanie komend sterujących (commands) i statusów urządzeń (telemetry) z gwarancją dostarczenia pakietów dzięki mechanizmom QoS (Quality of Service).
- **Warstwa Wykonawcza i Brzegowa (Edge Actuator Layer):** Obejmuje mikrokontrolery ESP32, które pełnią rolę inteligentnych sterowników oraz węzłów sensorycznych. Wykonują one fizyczne czynności (np. sterowanie serwomechanizmem szlabanu), obsługują interfejsy płatnicze (NFC/Pulse) oraz integrują czujniki ultradźwiękowe pełniące rolę sprzętowych wyzwalaczy (ang. hardware trigger) procesu detekcji.

Przepływ danych (Data Flow) Architektura wymusza jednokierunkowy, uporządkowany przepływ informacji:

1. Czujnik ultradźwiękowy podłączony do ESP32 wykrywa fizyczną obecność pojazdu przed szlabanem i publikuje komunikat wyzwalający (trigger) na brokerze MQTT.
2. Vision Engine, po odebraniu sygnału, rozpoczyna przechwytywanie i analizę klatek ze strumienia RTSP.
3. Vision Engine po odczytaniu tablicy wysyła zapytanie SQL do bazy PostgreSQL.
4. Po autoryzacji, serwer publikuje komunikat na odpowiednim temacie MQTT.
5. ESP32 odbiera komunikat i wykonuje akcję fizyczną, odsyłając status wykonania zadania do serwera.

Separacja tych warstw gwarantuje, że wymiana sprzętu (np. inna kamera lub szlaban) nie wymaga modyfikacji algorytmów głębokiego uczenia, co jest kluczową przewagą nad rozwiązaniami typu All-in-One.

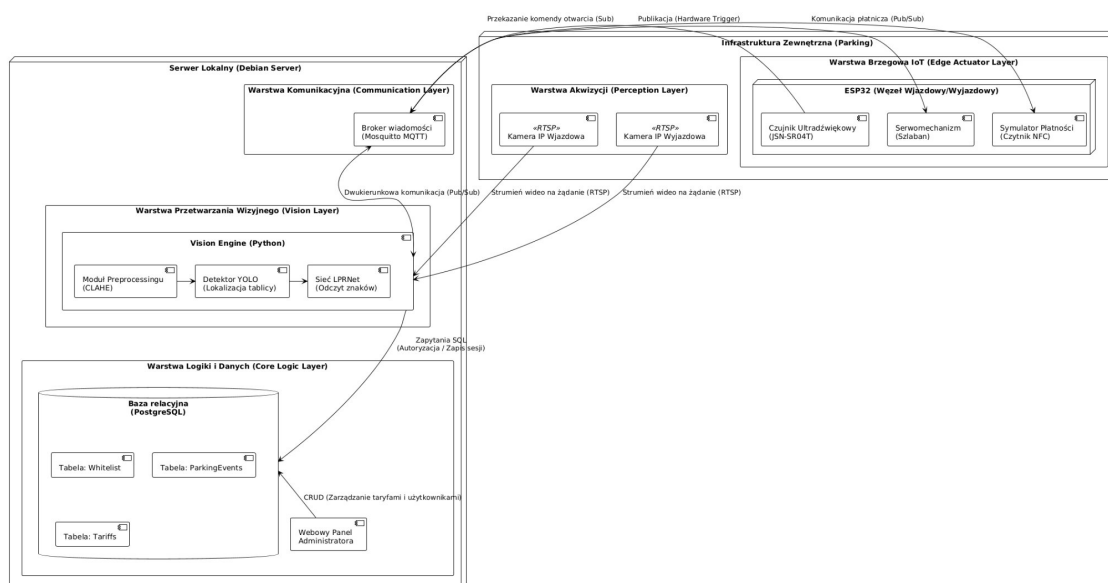


Diagram komponentów i wdrożenia

3.2. Konfiguracja środowiska serwerowego na platformie Debian Server

3.3. Opracowanie schematu i wdrożenie relacyjnej bazy danych PostgreSQL

Infrastruktura danych została zaimplementowana w PostgreSQL, zapewniając pełną integralność transakcyjną. Aby obsłużyć logikę darmowych wjazdów oraz płatnych sesji dla gości, schemat bazy danych (ERD) oparto na następujących tabelach:

- AdminUsers – przechowuje dane autoryzacyjne operatorów panelu webowego. Kolumny: user_id (PK), username, password_hash.
- Whitelist – rejestr pojazdów uprawnionych do stałego, bezpłatnego wjazdu (np. pracownicy, zaopatrzenie). Kolumny: whitelist_id (PK), license_plate (UNIQUE), description (opcjonalny opis/właściciel), is_active (boolean pozwalający na tymczasowe zablokowanie).
- Tariffs – definicja stawek dla pojazdów komercyjnych (nieznajdujących się na białej liście). Kolumny: tariff_id (PK), tariff_name, hourly_rate.
- ParkingEvents – główna ewidencja zdarzeń i sesji parkingowych dla wszystkich pojazdów. Kolumny: event_id (PK), license_plate (zapisywana jako tekst, aby obsłużyć auta niezarejestrowane w systemie), entry_time (timestamp wjazdu), exit_time (timestamp wyjazdu), calculated_fee (zapisywana przy wyjeździe, dla whitelisty wynosi 0), is_paid (boolean).



3.4. Akwizycja strumienia wideo i implementacja preprocessingu obrazu w środowisku Python

W celu optymalizacji obciążenia jednostki centralnej (CPU) oraz eliminacji zbędnego zużycia zasobów, zrezygnowano z ciągłej analizy strumienia wideo z kamer IP (tzw. busy-waiting). W systemie zaimplementowano architekturę sterowaną zdarzeniami z wykorzystaniem sprzętowego wyzwalacza (ang. hardware trigger).

Gdy pojazd podjeżdża do szlabanu, jego obecność jest rejestrowana przez zewnętrzny czujnik odległości, który za pośrednictwem protokołu MQTT wysyła sygnał wybudzający do jednostki serwerowej. Dopiero w tym momencie aplikacja napisana w języku Python łączy się ze strumieniem RTSP kamery, pobierając zbuforowaną sekwencję klatek wideo do analizy. Następnie na pobranym obrazie realizowany jest preprocessing z użyciem algorytmu CLAHE. Technika ta adaptacyjnie poprawia lokalny kontrast obrazu, redukując szумы w obszarach niedoświetlonych i niwelując refleksy świetlne z reflektorów pojazdu, co bezpośrednio podnosi skuteczność detekcji przez sieć YOLO w kolejnym etapie przetwarzania.

3.5. Integracja modeli głębokiego uczenia dla detekcji i odczytu tablic

3.6. Budowa warstwy sprzętowej IoT i programowanie mikrokontrolera ESP32

Warstwa sprzętowa prototypu bazuje na mikrokontrolerze ESP32, który dzięki zintegrowanym układom komunikacji bezprzewodowej (Wi-Fi) doskonale sprawdza się w systemach IoT. Na potrzeby środowiska testowego do mikrokontrolera podłączono trzy kluczowe podsystemy: serwomechanizm symulujący pracę szlabanu, czytnik kart zbliżeniowych NFC realizujący symulację płatności oraz wodoodporny moduł ultradźwiękowy JSN-SR04T (wersja AJ-SR04M).

Moduł JSN-SR04T został zaimplementowany jako detektor obecności pojazdu. Wybór tego konkretnego sensora podyktowany był jego hermetyczną konstrukcją (sonda zintegrowana w wodoodpornej obudowie), co w przeciwieństwie do standardowych czujników HC-SR04 pozwala na niezawodną pracę w warunkach zewnętrznych, odporną na wilgoć i zabrudzenia. Oprogramowanie mikrokontrolera dokonuje ciągłych pomiarów odległości z wykorzystaniem fali akustycznej. Kiedy zmierzony dystans spada poniżej zdefiniowanego programowo progu (co oznacza fizyczne naruszenie strefy przed szlabanem), ESP32 bezzwłocznie publikuje na brokerze MQTT komunikat triggerujący. Rozwiązanie to całkowicie zdejmuje z serwera analitycznego konieczność ciągłego poszukiwania pojazdów na obrazie z kamer, delegując to zadanie do wysoce energooszczędnej warstwy brzegowej.

3.7. Konfiguracja brokera MQTT i implementacja komunikacji asynchronicznej

3.8. Implementacja logiki biznesowej i webowego interfejsu administratora

Logika biznesowa zarządza przepływem danych między warstwą wizyjną a wykonawczą, rozróżniając pojazdy na podstawie przynależności do białej listy. Zrealizowano to za pomocą następujących przypadków użycia:

- **Wjazd pojazdów autoryzowanych** – proces inicjowany jest przez czujnik ultradźwiękowy (JSN-SR04T), który zgłasza obecność obiektu przed barierą. Potok wizyjny zostaje wybudzony, a po wyekstrahowaniu numeru license_plate (YOLO + LPRNet), system przeszukuje tabelę Whitelist. Jeśli numer figuruje w bazie i ma status is_active równy TRUE, serwer automatycznie publikuje komendę MQTT do ESP32. Szlaban zostaje otwarty, a w tabeli ParkingEvents tworzony jest wpis z czasem wjazdu i domyślnie zablokowanym naliczaniem opłat (calculated_fee = 0).

- **Paid Access & Payment Simulation (Obsługa gości komercyjnych)** – w przypadku braku odczytanej tablicy w tabeli Whitelist, po aktywacji czujnika i analizie obrazu, system traktuje pojazd jako klienta płatnego:
 - **Wjazd:** System otwiera szlaban wjazdowy (MQTT) i inicjuje nową sesję w ParkingEvents, logując numer rejestracyjny oraz entry_time.
 - **Wyjazd:** Po podjechaniu pojazdu do strefy wyjazdowej, czujnik ultradźwiękowy ponownie wyzwala system. Kamera wyjazdowa skanuje tablicę. System odszukuje otwartą sesję w ParkingEvents, oblicza czas postoju i na podstawie hourly_rate z tabeli Tariffs wylicza opłatę. Użytkownik korzysta z symulatora sprzętowego ESP32 – po zbliżeniu karty NFC saldo zostaje zweryfikowane i pomniejszone o wyliczoną kwotę (symulacja protokołu Pulse dla terminala Nayax). Po poprawnym odczycie, system aktualizuje rekord (is_paid = TRUE, loguje exit_time) i wysyła sygnał MQTT do otwarcia szlabanu wyjazdowego.
- **System Administration** – panel webowy umożliwia administratorowi dynamiczne zarządzanie uprawnieniami. Główną funkcjonalnością jest dodawanie, modyfikowanie i usuwanie rekordów w tabeli Whitelist, aktualizacja stawek w Tariffs oraz wgląd w pełną historię sesji parkingowych z tabeli ParkingEvents (w tym audyt nieopłaconych wyjazdów i problemów z detekcją).

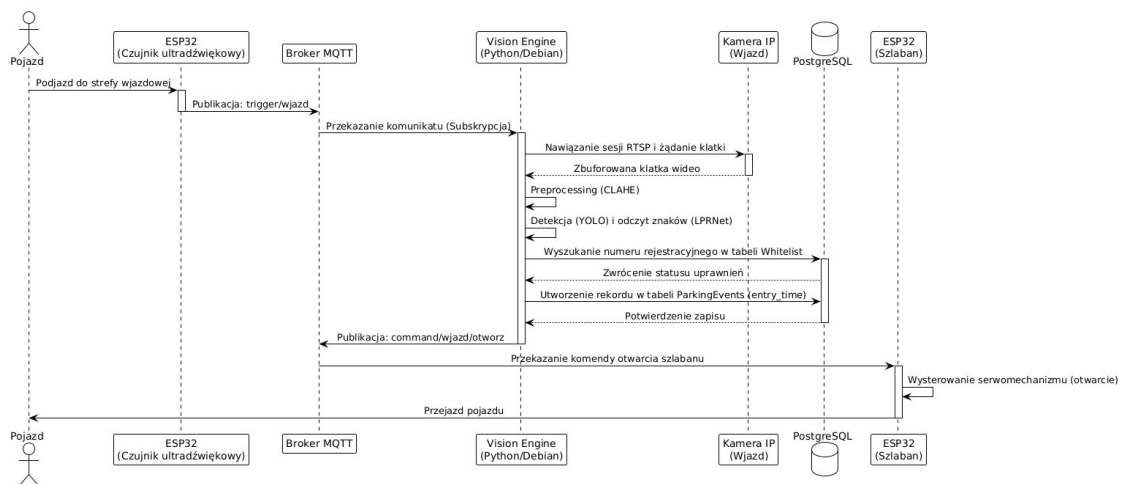


Diagram sekwencji dla wjazdu

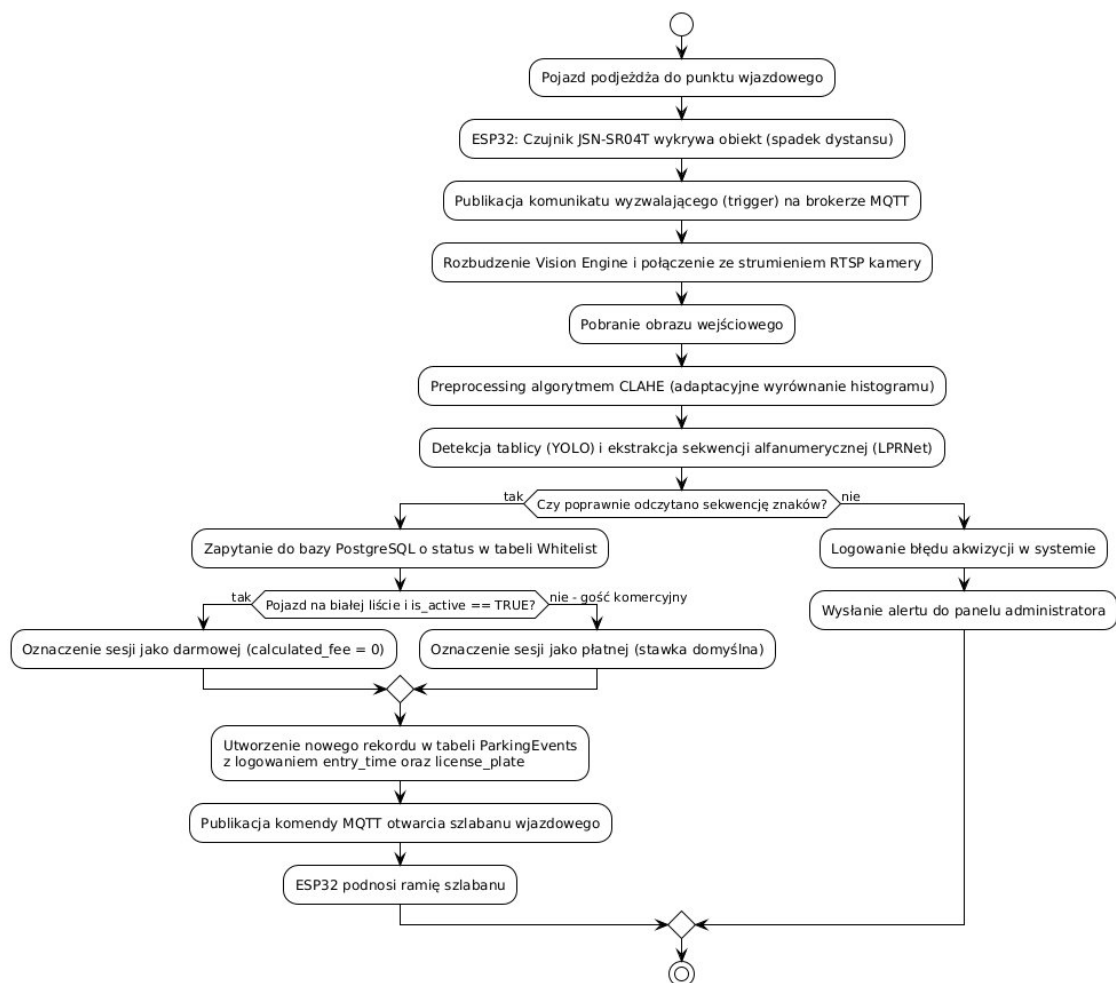


Diagram aktywności dla wjazdu

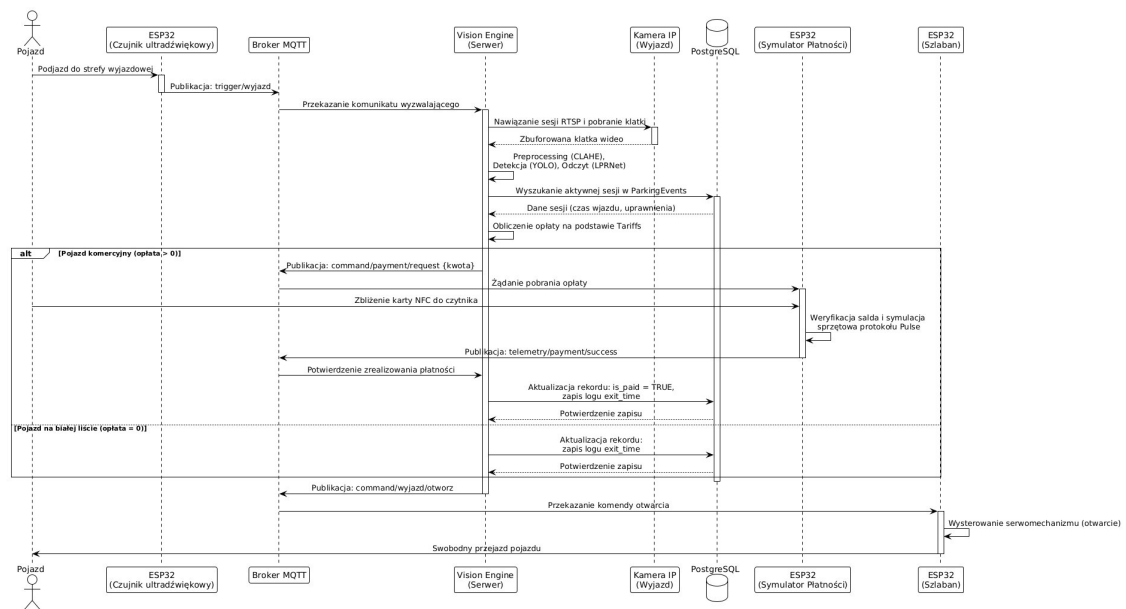


Diagram sekwencji dla wyjazdu

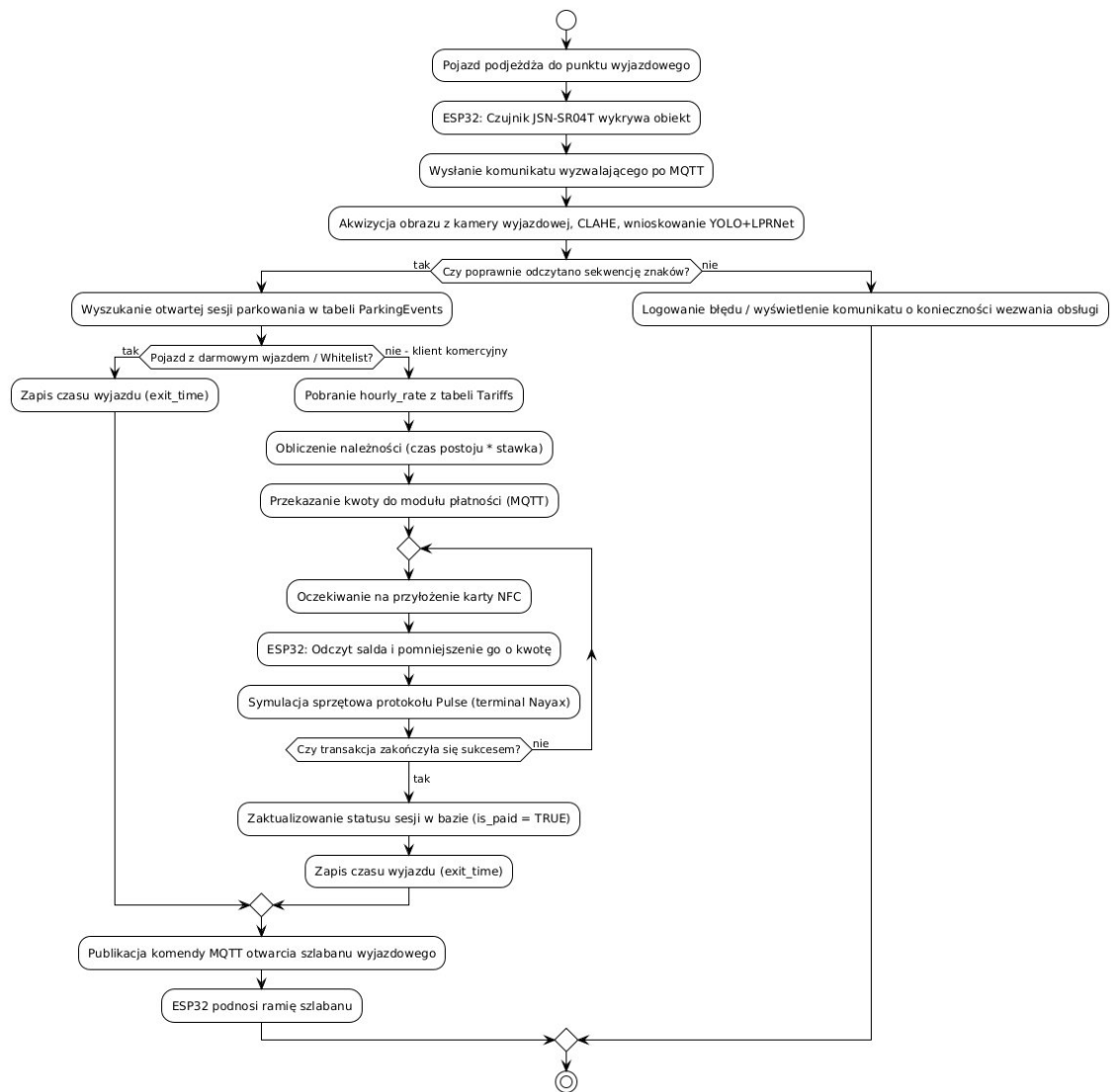


Diagram aktywności dla wyjazdu

4. WYNIKI BADAŃ I TESTY WYDAJNOŚCIOWE

4.1. Definicja scenariuszy oraz środowiska testowego

4.2. Skuteczność detekcji i odczytu tablic w warunkach pełnego oświetlenia

4.3. Analiza wpływu algorytmu CLAHE na poprawność odczytu w warunkach nocnych

4.4. Pomiary opóźnień systemu – latencja komunikacji

4.5. Stabilność działania w architekturze scentralizowanej

ZAKOŃCZENIE I WNIOSKI KOŃCOWE

BIBLIOGRAFIA

- [1] S. Roj-Rojewski, A. Porowski, Parkingi w przestrzeni miejskiej na przykładzie osiedla Centrum w Białymstoku, *Studia Miejskie*, t. 50, 2025, s. 47–59.
- [2] Polski Związek Przemysłu Motoryzacyjnego, Raport Branży Motoryzacyjnej 2025/2026, Warszawa, 2026.
- [3] OpenCV Team, *Cascade Classifier — OpenCV 4.x Documentation*, OpenCV, dostęp: https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html
- [4] G. Jocher, J. Qiu, *Ultralytics YOLO11*, wersja 11.0.0, 2024, dostęp: <https://github.com/ultralytics/ultralytics>
- [5] Tesseract OCR Team, *Improving the quality of the output*, Google, 2024, dostęp: <https://tesseract-ocr.github.io/tessdoc/ImproveQuality.html>
- [6] S. Zherzdev, A. S. Gruzdev, *LPRNet: License Plate Recognition via Deep Neural Networks*, CoRR, vol. abs/1806.10447, 2018, dostęp: <http://arxiv.org/abs/1806.10447>
- [7] OpenCV Team, *Histogram Equalization and CLAHE*, OpenCV, dostęp: https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html
- [8] OASIS Standard, *MQTT Version 5.0. OASIS Standard*, dostęp: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [9] Green Center s.r.o., *Systemy Parkingowe — Rozwiązania kompleksowe*, dostęp: <https://www.green.cz/pl>
- [10] City Parking Group, *Parkomatic — Systemy zarządzania parkowaniem*, dostęp: <https://cityparkinggroup.pl>
- [11] Smart2Park, *System SWMP: System Wspomagania Miejscami Parkingowymi*, dostęp: <https://www.smart2park.pl/2-system-swmp/>
- [12] CarFlow System, *Oferta systemów parkingowych*, dostęp: <https://carflowsystem.pl/oferta>
- [13] SmartOffice Expert, *Smart Park — Aplikacja do rezerwacji miejsc*, dostęp: <https://smartoffice.expert/pl/smart-park/>
- [14] Assecod Data Systems, *Strefy Parkowania — Rozwiązania Smart City*, dostęp: <https://www.assecods.pl/oferta/smart-city/strefy-parkowania>
- [15] SPIE Building Solutions, *Smart Parking — Oferta i rozwiązania*, dostęp: <https://www.spie.com/pl/poznaj-nasze-oferty-rozwiazania-i-doswiadczenie/nasze-oferty/smart-parking>
- [16] M. Hill, *OpenALPR: Automatic License Plate Recognition Library*, GitHub, dostęp: <https://github.com/openalpr/openalpr>
- [17] Nomeroff Net Team, *Nomeroff Net: Automatic License Plate Recognition System*, GitHub, dostęp: <https://github.com/ria-com/nomeroff-net>
- [18] ParkPow, *Plate Recognizer: Cloud ALPR API & On-Premise SDK*, dostęp: <https://platerecognizer.com>
- [19] Bevywise Networks, *MQTT vs REST: Choosing the Right Protocol for IoT Implementation*, dostęp: <https://www.bevywise.com/blog/mqtt-vs-rest-iot-implementation/>

[20] A. Author (PMC), Deep Learning-Based Approaches for License Plate Recognition, National Center for Biotechnology Information (NCBI), dostęp:

<https://pmc.ncbi.nlm.nih.gov/articles/PMC12758793/>

[21] Nayax Ltd., Understanding Attributes and Configurations — Pulse Protocol, Nayax University, dostęp:

<https://nayax-u.nayax.com/scenario/understanding-attributes-and-configurations-pulse-11715>

WYKAZ FOTOGRAFII

WYKAZ RYSUNKÓW

WYKAZ TABEL

WYKAZ WYKRESÓW

STRESZCZENIE

Przedmiotem pracy jest zaprojektowanie i budowa prototypu zintegrowanego systemu klasy Smart Parking, którego głównym zadaniem jest automatyzacja procesu kontroli dostępu za pomocą technologii rozpoznawania tablic rejestracyjnych (ALPR). Projekt rozwiązuje problem niskiej przepustowości tradycyjnych systemów parkingowych oraz ogranicza koszty eksploatacji poprzez eliminację fizycznych nośników uprawnień (biletów, kart).

System opiera się na architekturze scentralizowanej, w której obliczeniowo wymagające procesy analizy obrazu są odseparowane od urządzeń brzegowych. Sercem systemu jest serwer pracujący w środowisku Linux, realizujący potok przetwarzania obrazu (Vision Engine) oparty na języku Python. Proces detekcji i rozpoznawania numerów rejestracyjnych realizowany jest hybrydowo: sieć neuronowa YOLO odpowiada za lokalizację tablicy na obrazie ze strumienia RTSP, natomiast sieć LPRNet dokonuje predykcji znaków. W celu zapewnienia niezawodności systemu w zmiennych warunkach oświetleniowych, zaimplementowano preprocessing obrazu wykorzystujący algorytm CLAHE.

Warstwa komunikacyjna z infrastrukturą fizyczną (model szlabanu) realizowana jest w oparciu o protokół MQTT. Wykorzystanie mikrokontrolera ESP32 jako jednostki sterującej pozwala na szybką reakcję na komendy serwera przy zachowaniu wysokiej stabilności połączenia (mechanizmy LWT oraz QoS). Logika biznesowa systemu, zarządzana przez relacyjną bazę danych PostgreSQL, umożliwia elastyczne różnicowanie taryf oraz automatyczną ewidencję zdarzeń dla różnych grup użytkowników (pracownicy, goście).

Efektem końcowym pracy będzie działający prototyp w małej skali, integrujący zaawansowane algorytmy uczenia głębokiego z systemami wbudowanymi i aplikacją webową do zarządzania systemem, z możliwością implementacji na dużą skalę np. na Uniwersytecie